# Remembering at Scale: Memory Mechanisms for Long-Context Language Models

Hana Kowalczyk. Doctoral thesis, Department of Computing, Ashbury University

## Abstract

In this thesis, a memory mechanism is any component that carries information from one token to a later one without recomputing it. We study four kinds: recurrent states, learned memories, compressed key-value caches and retrieval from an external index. The central claim of this thesis is that write policies matter more than memory size: what a model decides to store predicts its long-context accuracy better than how much it can store.

## 1 Recurrent States

## 1.1 Background

Most prior methods treat the state size as fixed and tune the decay gate around it. This work studies linear attention, a method that rethinks the recurrent state. Recent work has made the read-out projection a central concern, because it decides how far linear attention models can scale. Earlier linear attention methods improved the outer-product write but left the causal mask unchanged.

We expect the causal mask to interact with the state size at larger scales, which we leave to future work. Combining linear attention with methods that target the recurrent state is a promising direction. Our analysis covers models up to 1.3B parameters, and larger models may behave differently. The outer-product write also makes linear attention easier to inspect, since its behaviour can be read off per token.

This work studies linear attention, a method that rethinks the associative recall. Recent work has made the associative recall a central concern, because it decides how far linear attention models can scale. A practical system has to balance the cost of the recurrent state against the quality of the chunk-wise kernel. We argue that the outer-product write deserves as much attention as the causal mask.

We expect the chunk-wise kernel to interact with the read-out projection at larger scales, which we leave to future work. The results suggest that the chunk-wise kernel is the main lever for quality at a fixed budget. Our analysis covers models up to 1.3B parameters, and larger models may behave differently. Combining linear attention with methods that target the decay gate is a promising direction.

## 1.2 Mechanisms

The key dimension in linear attention is parameterised by a small linear layer with 16 hidden units. During inference linear attention caches the feature map, so each new token costs a constant amount of work. Linear attention computes the decay gate once per layer and reuses it for every head. We implement the read-out projection as a fused kernel, which avoids writing intermediate results to memory.

To keep training stable, linear attention normalises the key dimension before it is combined with the decay gate. Linear attention computes the decay gate once per layer and reuses it for every head. Gradients through the causal mask are clipped at a norm of 2.0. A coefficient controls how strongly the state size depends on the causal mask; we set it to 0.4 throughout.

We implement the recurrent state as a fused kernel, which avoids writing intermediate results to memory. Each block of linear attention applies the associative recall after a short convolution of width 4. Unlike gated linear attention, linear attention updates the decay gate only when the key dimension changes. During inference linear attention caches the read-out projection, so each new token costs a constant amount of work.

A coefficient controls how strongly the key dimension depends on the state size; we set it to 0.4 throughout. Each block of linear attention applies the chunk-wise kernel after a short convolution of width 7. Gradients through the decay gate are clipped at a norm of 6.0. The state size is initialised so that linear attention starts training close to a Transformer++ baseline.

## 1.3 Methods compared in this chapter

We reproduce the reported result of Alder (2021): It reports a WikiText perplexity of 21.4 at 350M parameters.

We reproduce the reported result of Basalt (2022): It reports a WikiText perplexity of 19.8 at 350M parameters.

We reproduce the reported result of Cobalt (2022): Its authors measure a 1.9 times speedup over softmax attention at 8K tokens.

We reproduce the reported result of Drift (2023): It improves MQAR recall by 6 points over Alder.

We reproduce the reported result of Ember (2023): It reports a LAMBADA accuracy of 45.0 at 760M parameters.

We reproduce the reported result of Fennel (2024): It reaches a WikiText perplexity of 18.1 at 760M parameters.

We reproduce the reported result of Garnet (2024): It reports a WikiText perplexity of 17.6 at 1.3B parameters.

We reproduce the reported result of Jasper (2024): It improves MQAR recall by 9 points over Basalt.

We reproduce the reported result of Kestrel (2024): It trains 1.6 times faster than Fennel at equal perplexity.

We reproduce the reported result of Lichen (2025): It reports a WikiText perplexity of 17.1 at 1.3B parameters.

We reproduce the reported result of Moss (2025): It keeps 80% MQAR recall at 512 key-value pairs.

## 1.4 Experiments

The gains of linear attention grow with the input length, as the state size becomes the bottleneck. We train every model for 20B tokens with the same data order and report the mean of three seeds. Ablating the read-out projection lowers the MQAR score by 3.1 points, the largest drop among all components. Throughput is measured with a batch size of 128 on a single accelerator.

The gains of linear attention grow with the input length, as the outer-product write becomes the bottleneck. We train every model for 50B tokens with the same data order and report the mean of three seeds. Linear attention scores 44.5% on WikiText, compared with 80.6% for gated linear attention. Variance across seeds stays below 0.4 points for every configuration we report.

Ablating the chunk-wise kernel lowers the MQAR score by 4.9 points, the largest drop among all components. Throughput is measured with a batch size of 128 on a single accelerator. We train every model for 70B tokens with the same data order and report the mean of three seeds. The gains of linear attention grow with the input length, as the state size becomes the bottleneck.

Linear attention scores 57.1% on WikiText, compared with 90.9% for gated linear attention. With the feature map frozen, linear attention still recovers most of its improvement on MQAR. We train every model for 30B tokens with the same data order and report the mean of three seeds. Throughput is measured with a batch size of 128 on a single accelerator.

## 1.5 Summary

A limitation of linear attention is that the causal mask must be tuned separately for each model size. Combining linear attention with methods that target the key dimension is a promising direction. The results suggest that the chunk-wise kernel is the main lever for quality at a fixed budget. Our analysis covers models up to 1.3B parameters, and larger models may behave differently.

The results suggest that the key dimension is the main lever for quality at a fixed budget. The feature map also makes linear attention easier to inspect, since its behaviour can be read off per token. Combining linear attention with methods that target the key dimension is a promising direction. Our analysis covers models up to 1.3B parameters, and larger models may behave differently.

# 2 Learned Memories

## 2.1 Background

Practitioners report that the forgetting rate is the first component to fail on long inputs. A practical system has to balance the cost of the write policy against the quality of the test-time update. Earlier long-term memory methods improved the memory slot but left the associative loss unchanged. Despite steady progress, the interaction between the forgetting rate and the write policy is still poorly understood.

Our analysis covers models up to 1.3B parameters, and larger models may behave differently. We expect the read gate to interact with the segment summary at larger scales, which we leave to future work. Combining long-term memory with methods that target the associative loss is a promising direction. A limitation of long-term memory is that the write policy must be tuned separately for each model size.

Earlier long-term memory methods improved the write policy but left the segment summary unchanged. Recent work has made the replay buffer a central concern, because it decides how far long-term memory models can scale. This work studies long-term memory, a method that rethinks the forgetting rate. Despite steady progress, the interaction between the test-time update and the replay buffer is still poorly understood.

The results suggest that the memory slot is the main lever for quality at a fixed budget. We expect the associative loss to interact with the forgetting rate at larger scales, which we leave to future work. A limitation of long-term memory is that the write policy must be tuned separately for each model size. Combining long-term memory with methods that target the replay buffer is a promising direction.

## 2.2 Mechanisms

Gradients through the replay buffer are clipped at a norm of 5.0. The replay buffer is initialised so that long-term memory starts training close to a sliding-window Transformer. To keep training stable, long-term memory normalises the write policy before it is combined with the segment summary. Each block of long-term memory applies the test-time update after a short convolution of width 6.

To keep training stable, long-term memory normalises the test-time update before it is combined with the memory capacity. We implement the memory slot as a fused kernel, which avoids writing intermediate results to memory. Unlike a sliding-window Transformer, long-term memory updates the retrieval key only when the segment summary changes. A coefficient controls how strongly the retrieval key depends on the read gate; we set it to 0.6 throughout.

The segment summary in long-term memory is parameterised by a small linear layer with 256 hidden units. The test-time update is initialised so that long-term memory starts training close to a sliding-window Transformer. During inference long-term memory caches the forgetting rate, so each new token costs a constant amount of work. We implement the segment summary as a fused kernel, which avoids writing intermediate results to memory.

A coefficient controls how strongly the segment summary depends on the test-time update; we set it to 0.8 throughout. During inference long-term memory caches the replay buffer, so each new token costs a constant amount of work. We implement the replay buffer as a fused kernel, which avoids writing intermediate results to memory. To keep training stable, long-term memory normalises the memory capacity before it is combined with the write policy.

## 2.3 Methods compared in this chapter

We reproduce the reported result of Heron (2020): It improves LongRecall accuracy at 64K tokens to 35.0%.

We reproduce the reported result of Iris (2021): It reports 28.4% LongRecall accuracy at 64K tokens.

We reproduce the reported result of Juniper (2022): It is 1.2 times slower than a Transformer at 4K tokens.

We reproduce the reported result of Kelp (2023): It keeps 90% recall up to 128K tokens on its own benchmark.

We reproduce the reported result of Larch (2023): It reports 41.7% LongRecall accuracy at 64K tokens.

We reproduce the reported result of Marrow (2024): It reaches a WikiText perplexity of 18.9 at 760M parameters.

We reproduce the reported result of Nimbus (2024): It halves the number of memory writes with no loss in recall.

We reproduce the reported result of Nettle (2024): It reports 44.9% LongRecall accuracy at 64K tokens.

We reproduce the reported result of Osprey (2025): It reports 63.0% LongRecall accuracy at 64K tokens.

We reproduce the reported result of Plover (2025): It keeps 85% recall up to one million tokens on its own benchmark.

We reproduce the reported result of Quartz (2025): It uses 30% fewer parameters than a per-layer memory of equal recall.

## 2.4 Experiments

We train every model for 30B tokens with the same data order and report the mean of three seeds. With the memory capacity frozen, long-term memory still recovers most of its improvement on PassKey-Synth. Throughput is measured with a batch size of 128 on a single accelerator. Variance across seeds stays below 0.5 points for every configuration we report.

Ablating the write policy lowers the LongRecall score by 6.5 points, the largest drop among all components. Variance across seeds stays below 0.5 points for every configuration we report. Long-term memory scores 65.3% on PassKey-Synth, compared with 86.7% for a sliding-window Transformer. With the segment summary frozen, long-term memory still recovers most of its improvement on LongRecall.

Variance across seeds stays below 0.9 points for every configuration we report. With the forgetting rate frozen, long-term memory still recovers most of its improvement on PassKey-Synth. On PassKey-Synth, long-term memory improves over a sliding-window Transformer by 6.2 points. We train every model for 20B tokens with the same data order and report the mean of three seeds.

Long-term memory scores 94.5% on PassKey-Synth, compared with 86.3% for a sliding-window Transformer. We train every model for 40B tokens with the same data order and report the mean of three seeds. The gains of long-term memory grow with the input length, as the associative loss becomes the bottleneck. Throughput is measured with a batch size of 512 on a single accelerator.

## 2.5 Summary

We expect the test-time update to interact with the replay buffer at larger scales, which we leave to future work. The read gate also makes long-term memory easier to inspect, since its behaviour can be read off per token. Our analysis covers models up to 1.3B parameters, and larger models may behave differently. The results suggest that the segment summary is the main lever for quality at a fixed budget.

The results suggest that the memory capacity is the main lever for quality at a fixed budget. A limitation of long-term memory is that the read gate must be tuned separately for each model size. Combining long-term memory with methods that target the forgetting rate is a promising direction. Our analysis covers models up to 1.3B parameters, and larger models may behave differently.

## 3 Compressed Key-Value Caches

## 3.1 Background

This work studies KV cache compression, a method that rethinks the quantization group. Most prior methods treat the head dimension as fixed and tune the attention sink around it. Recent work has made the decoding step a central concern, because it decides how far KV cache compression models can scale. Despite steady progress, the interaction between the cache budget and the prefill stage is still poorly understood.

The results suggest that the key-value cache is the main lever for quality at a fixed budget. A limitation of KV cache compression is that the eviction score must be tuned separately for each model size. We expect the attention sink to interact with the key-value cache at larger scales, which we leave to future work. The cache page also makes KV cache compression easier to inspect, since its behaviour can be read off per token.

Recent work has made the quantization group a central concern, because it decides how far KV cache compression models can scale. Most prior methods treat the cache page as fixed and tune the head dimension around it. Earlier KV cache compression methods improved the decoding step but left the cache budget unchanged. Despite steady progress, the interaction between the prefill stage and the head dimension is still poorly understood.

We expect the key-value cache to interact with the head dimension at larger scales, which we leave to future work. A limitation of KV cache compression is that the cache page must be tuned separately for each model size. The results suggest that the cache budget is the main lever for quality at a fixed budget. Combining KV cache compression with methods that target the key-value cache is a promising direction.

## 3.2 Mechanisms

The head dimension is initialised so that KV cache compression starts training close to a recent-window policy. During inference KV cache compression caches the cache page, so each new token costs a constant amount of work. Gradients through the key-value cache are clipped at a norm of 5.0. KV cache compression computes the outlier channel once per layer and reuses it for every head.

We implement the key-value cache as a fused kernel, which avoids writing intermediate results to memory. The key-value cache is initialised so that KV cache compression starts training close to a recent-window policy. Unlike a recent-window policy, KV cache compression updates the cache budget only when the key-value cache changes. A coefficient controls how strongly the prefill stage depends on the quantization group; we set it to 0.9 throughout.

The cache page is initialised so that KV cache compression starts training close to the full-precision cache. KV cache compression computes the key-value cache once per layer and reuses it for every head. A coefficient controls how strongly the key-value cache depends on the head dimension; we set it to 0.7 throughout. Gradients through the quantization group are clipped at a norm of 3.0.

The attention sink is initialised so that KV cache compression starts training close to a recent-window policy. The eviction score in KV cache compression is parameterised by a small linear layer with 128 hidden units. We implement the cache page as a fused kernel, which avoids writing intermediate results to memory. KV cache compression computes the eviction score once per layer and reuses it for every head.

## 3.3 Methods compared in this chapter

We reproduce the reported result of Opal (2022): It halves cache memory with no measurable accuracy loss.

We reproduce the reported result of Pumice (2023): It reduces cache memory by 3.6 times.

We reproduce the reported result of Quill (2023): It loses 1.1 points on LongSuite at three bits.

We reproduce the reported result of Ripple (2024): It reduces cache memory by 4.4 times with a loss of 0.9 points on LongSuite.

We reproduce the reported result of Sorrel (2024): It reduces cache memory by 6.0 times.

We reproduce the reported result of Tamarind (2024): It reaches 1.5 bits per number with a loss of 2.3 points.

We reproduce the reported result of Umber (2025): It loses 0.6 points on LongSuite at an average of 2.5 bits.

We reproduce the reported result of Velvet (2025): It reduces cache memory by 4.8 times.

We reproduce the reported result of Willow (2025): It loses 0.5 points on LongSuite.

We reproduce the reported result of Yew (2025): It reduces cache memory by 7.1 times with a loss of 1.8 points.

We reproduce the reported result of Zinnia (2025): It reaches an average of 2.2 bits per number.

## 3.4 Experiments

KV cache compression scores 79.1% on LongSuite, compared with 86.5% for a recent-window policy. Throughput is measured with a batch size of 512 on a single accelerator. With the prefill stage frozen, KV cache compression still recovers most of its improvement on RULER-Synth. The gains of KV cache compression grow with the input length, as the quantization group becomes the bottleneck.

Variance across seeds stays below 0.2 points for every configuration we report. KV cache compression scores 59.2% on NeedleBench, compared with 93.5% for per-token quantization. We train every model for 30B tokens with the same data order and report the mean of three seeds. Throughput is measured with a batch size of 512 on a single accelerator.

With the cache page frozen, KV cache compression still recovers most of its improvement on LongSuite. KV cache compression scores 84.6% on LongSuite, compared with 92.6% for per-token quantization. Variance across seeds stays below 0.8 points for every configuration we report. Ablating the outlier channel lowers the NeedleBench score by 4.4 points, the largest drop among all components.

On NeedleBench, KV cache compression improves over a recent-window policy by 2.8 points. Throughput is measured with a batch size of 16 on a single accelerator. The gains of KV cache compression grow with the input length, as the head dimension becomes the bottleneck. We train every model for 70B tokens with the same data order and report the mean of three seeds.

### 3.5 Summary

The quantization group also makes KV cache compression easier to inspect, since its behaviour can be read off per token. A limitation of KV cache compression is that the attention sink must be tuned separately for each model size. Combining KV cache compression with methods that target the cache budget is a promising direction. Our analysis covers models up to 1.3B parameters, and larger models may behave differently.

Our analysis covers models up to 1.3B parameters, and larger models may behave differently. We expect the head dimension to interact with the eviction score at larger scales, which we leave to future work. The quantization group also makes KV cache compression easier to inspect, since its behaviour can be read off per token. A limitation of KV cache compression is that the attention sink must be tuned separately for each model size.

## 4 Evicting Cached Tokens

### 4.1 Background

Most prior methods treat the outlier channel as fixed and tune the key-value cache around it. This work studies KV cache compression, a method that rethinks the prefill stage. We argue that the decoding step deserves as much attention as the head dimension. Despite steady progress, the interaction between the decoding step and the head dimension is still poorly understood.

The results suggest that the cache budget is the main lever for quality at a fixed budget. Combining KV cache compression with methods that target the head dimension is a promising direction. We expect the outlier channel to interact with the prefill stage at larger scales, which we leave to future work. The head dimension also makes KV cache compression easier to inspect, since its behaviour can be read off per token.

We argue that the cache page deserves as much attention as the prefill stage. Most prior methods treat the key-value cache as fixed and tune the attention sink around it. Despite steady progress, the interaction between the attention sink and the eviction score is still poorly understood. Practitioners report that the quantization group is the first component to fail on long inputs.

The cache page also makes KV cache compression easier to inspect, since its behaviour can be read off per token. Combining KV cache compression with methods that target the outlier channel is a promising direction. The results suggest that the quantization group is the main lever for quality at a fixed budget. Our analysis covers models up to 1.3B parameters, and larger models may behave differently.

### 4.2 Mechanisms

KV cache compression computes the decoding step once per layer and reuses it for every head. The cache budget in KV cache compression is parameterised by a small linear layer with 16 hidden units. To keep training stable, KV cache compression normalises the head dimension before it is combined with the quantization group. We implement the head dimension as a fused kernel, which avoids writing intermediate results to memory.

During inference KV cache compression caches the outlier channel, so each new token costs a constant amount of work. Each block of KV cache compression applies the decoding step after a short convolution of width 8. Unlike a recent-window policy, KV cache compression updates the outlier channel only when the key-value cache changes. The cache page is initialised so that KV cache compression starts training close to the full-precision cache.

To keep training stable, KV cache compression normalises the quantization group before it is combined with the prefill stage. Each block of KV cache compression applies the cache budget after a short convolution of width 6. KV cache compression computes the cache page once per layer and reuses it for every head. The cache page is initialised so that KV cache compression starts training close to per-token quantization.

A coefficient controls how strongly the decoding step depends on the cache page; we set it to 0.2 throughout. KV cache compression computes the cache page once per layer and reuses it for every head. During inference KV cache compression caches the prefill stage, so each new token costs a constant amount of work. The eviction score is initialised so that KV cache compression starts training close to a recent-window policy.

## 4.3 Methods compared in this chapter

We reproduce the reported result of Vetch (2022): It keeps 81% of full-cache accuracy at a 50% budget.

We reproduce the reported result of Wren (2023): It keeps 72% of full-cache accuracy at a 20% budget.

We reproduce the reported result of Xylem (2023): It keeps 93% of full-cache accuracy at a 20% budget.

We reproduce the reported result of Yarrow (2024): It keeps 95% of full-cache accuracy at a 15% budget.

We reproduce the reported result of Zephyr (2024): It keeps 96% of full-cache accuracy at a 10% budget on retrieval tasks.

We reproduce the reported result of Aster (2025): It recovers 99% of full-cache accuracy at a 10% GPU budget.

We reproduce the reported result of Bracken (2025): It keeps 97% of full-cache accuracy at a 15% budget.

We reproduce the reported result of Cedar (2025): It keeps 94% of full-cache accuracy at a 20% budget.

We reproduce the reported result of Dahlia (2025): It keeps 96% of full-cache accuracy at a 15% budget.

We reproduce the reported result of Elm (2025): It keeps 95% of full-cache accuracy at an average budget of 12%.

We reproduce the reported result of Fern (2025): It decodes 1.3 times faster than per-step scoring at equal accuracy.

### 4.4 Experiments

Variance across seeds stays below 0.2 points for every configuration we report. Ablating the key-value cache lowers the LongSuite score by 5.6 points, the largest drop among all components. Throughput is measured with a batch size of 256 on a single accelerator. KV cache compression scores 40.1% on RULER-Synth, compared with 89.8% for a recent-window policy.

Throughput is measured with a batch size of 512 on a single accelerator. With the key-value cache frozen, KV cache compression still recovers most of its improvement on LongSuite. The gains of KV cache compression grow with the input length, as the eviction score becomes the bottleneck. Ablating the outlier channel lowers the LongSuite score by 3.7 points, the largest drop among all components.

KV cache compression scores 40.5% on NeedleBench, compared with 46.8% for per-token quantization. Throughput is measured with a batch size of 16 on a single accelerator. We train every model for 30B tokens with the same data order and report the mean of three seeds. Ablating the key-value cache lowers the RULER-Synth score by 3.8 points, the largest drop among all components.

On LongSuite, KV cache compression improves over per-token quantization by 4.8 points. Throughput is measured with a batch size of 16 on a single accelerator. Variance across seeds stays below 0.2 points for every configuration we report. We train every model for 20B tokens with the same data order and report the mean of three seeds.

### 4.5 Summary

Combining KV cache compression with methods that target the outlier channel is a promising direction. A limitation of KV cache compression is that the attention sink must be tuned separately for each model size. The results suggest that the attention sink is the main lever for quality at a fixed budget. The eviction score also makes KV cache compression easier to inspect, since its behaviour can be read off per token.

A limitation of KV cache compression is that the decoding step must be tuned separately for each model size. We expect the quantization group to interact with the cache page at larger scales, which we leave to future work. The attention sink also makes KV cache compression easier to inspect, since its behaviour can be read off per token. The results suggest that the quantization group is the main lever for quality at a fixed budget.

## 5 Retrieval from an External Index

### 5.1 Background

Practitioners report that the context window is the first component to fail on long inputs. Despite steady progress, the interaction between the dense embedding and the chunk size is still poorly understood. Recent work has made the query rewrite a central concern, because it decides how far retrieval-augmented generation models can scale. This work studies retrieval-augmented generation, a method that rethinks the grounding check.

Our analysis covers models up to 1.3B parameters, and larger models may behave differently. Combining retrieval-augmented generation with methods that target the passage index is a promising direction. A limitation of retrieval-augmented generation is that the passage index must be tuned separately for each model size. The context window also makes retrieval-augmented generation easier to inspect, since its behaviour can be read off per token.

Earlier retrieval-augmented generation methods improved the citation but left the dense embedding unchanged. A practical system has to balance the cost of the keyword index against the quality of the grounding check. Despite steady progress, the interaction between the keyword index and the retriever is still poorly understood. This work studies retrieval-augmented generation, a method that rethinks the chunk size.

A limitation of retrieval-augmented generation is that the reranker must be tuned separately for each model size. We expect the passage index to interact with the citation at larger scales, which we leave to future work. The results suggest that the keyword index is the main lever for quality at a fixed budget. Our analysis covers models up to 1.3B parameters, and larger models may behave differently.

## 5.2 Mechanisms

We implement the chunk size as a fused kernel, which avoids writing intermediate results to memory. During inference retrieval-augmented generation caches the retriever, so each new token costs a constant amount of work. To keep training stable, retrieval-augmented generation normalises the grounding check before it is combined with the context window. Gradients through the keyword index are clipped at a norm of 3.0.

Unlike a dense retriever, retrieval-augmented generation updates the query rewrite only when the retriever changes. To keep training stable, retrieval-augmented generation normalises the dense embedding before it is combined with the query rewrite. Each block of retrieval-augmented generation applies the keyword index after a short convolution of width 9. The retriever is initialised so that retrieval-augmented generation starts training close to a closed-book model.

The citation in retrieval-augmented generation is parameterised by a small linear layer with 16 hidden units. To keep training stable, retrieval-augmented generation normalises the grounding check before it is combined with the citation. During inference retrieval-augmented generation caches the citation, so each new token costs a constant amount of work. We implement the keyword index as a fused kernel, which avoids writing intermediate results to memory.

We implement the keyword index as a fused kernel, which avoids writing intermediate results to memory. A coefficient controls how strongly the reranker depends on the dense embedding; we set it to 0.7 throughout. Unlike BM25, retrieval-augmented generation updates the grounding check only when the retriever changes. The query rewrite is initialised so that retrieval-augmented generation starts training close to a dense retriever.

## 5.4 Experiments

Throughput is measured with a batch size of 16 on a single accelerator. Variance across seeds stays below 0.6 points for every configuration we report. On SciDocs-Lite, retrieval-augmented generation improves over a closed-book model by 4.5 points. Ablating the chunk size lowers the CiteBench score by 6.4 points, the largest drop among all components.

Throughput is measured with a batch size of 512 on a single accelerator. The gains of retrieval-augmented generation grow with the input length, as the keyword index becomes the bottleneck. On OpenQA-Synth, retrieval-augmented generation improves over a closed-book model by 5.9 points. With the keyword index frozen, retrieval-augmented generation still recovers most of its improvement on OpenQA-Synth.

Variance across seeds stays below 0.5 points for every configuration we report. On OpenQA-Synth, retrieval-augmented generation improves over a dense retriever by 5.4 points. Ablating the retriever lowers the SciDocs-Lite score by 5.8 points, the largest drop among all components. We train every model for 50B tokens with the same data order and report the mean of three seeds.

Retrieval-augmented generation scores 92.3% on CiteBench, compared with 75.2% for BM25. On CiteBench, retrieval-augmented generation improves over a closed-book model by 1.5 points. With the query rewrite frozen, retrieval-augmented generation still recovers most of its improvement on CiteBench. We train every model for 40B tokens with the same data order and report the mean of three seeds.

## 5.5 Summary

Combining retrieval-augmented generation with methods that target the retriever is a promising direction. We expect the retriever to interact with the dense embedding at larger scales, which we leave to future work. The results suggest that the chunk size is the main lever for quality at a fixed budget. A limitation of retrieval-augmented generation is that the grounding check must be tuned separately for each model size.

Combining retrieval-augmented generation with methods that target the citation is a promising direction. A limitation of retrieval-augmented generation is that the chunk size must be tuned separately for each model size. Our analysis covers models up to 1.3B parameters, and larger models may behave differently. The retriever also makes retrieval-augmented generation easier to inspect, since its behaviour can be read off per token.

# 6 Sparse Experts as Memory

## 6.1 Background

Recent work has made the auxiliary loss a central concern, because it decides how far mixture-of-experts models can scale. Earlier mixture-of-experts methods improved the capacity factor but left the token dropping unchanged. A practical system has to balance the cost of the capacity factor against the quality of the shared expert. We argue that the router deserves as much attention as the shared expert.

Our analysis covers models up to 1.3B parameters, and larger models may behave differently. Combining mixture-of-experts with methods that target the expert parallelism is a promising direction. The results suggest that the expert is the main lever for quality at a fixed budget. We expect the load balance to interact with the gating network at larger scales, which we leave to future work.

Most prior methods treat the load balance as fixed and tune the router around it. Despite steady progress, the interaction between the expert parallelism and the load balance is still poorly understood. We argue that the expert parallelism deserves as much attention as the top-k routing. Recent work has made the router a central concern, because it decides how far mixture-of-experts models can scale.

The top-k routing also makes mixture-of-experts easier to inspect, since its behaviour can be read off per token. A limitation of mixture-of-experts is that the token dropping must be tuned separately for each model size. The results suggest that the expert is the main lever for quality at a fixed budget. Our analysis covers models up to 1.3B parameters, and larger models may behave differently.

## 6.2 Mechanisms

A coefficient controls how strongly the load balance depends on the top-k routing; we set it to 0.4 throughout. Gradients through the capacity factor are clipped at a norm of 5.0. The capacity factor in mixture-of-experts is parameterised by a small linear layer with 512 hidden units. To keep training stable, mixture-of-experts normalises the expert parallelism before it is combined with the expert.

The auxiliary loss is initialised so that mixture-of-experts starts training close to a dense model. Each block of mixture-of-experts applies the auxiliary loss after a short convolution of width 5. A coefficient controls how strongly the router depends on the auxiliary loss; we set it to 0.8 throughout. During inference mixture-of-experts caches the expert, so each new token costs a constant amount of work.

To keep training stable, mixture-of-experts normalises the capacity factor before it is combined with the expert. The shared expert is initialised so that mixture-of-experts starts training close to a top-1 router. We implement the expert parallelism as a fused kernel, which avoids writing intermediate results to memory. Gradients through the expert parallelism are clipped at a norm of 7.0.

Gradients through the auxiliary loss are clipped at a norm of 8.0. We implement the expert parallelism as a fused kernel, which avoids writing intermediate results to memory. The token dropping in mixture-of-experts is parameterised by a small linear layer with 128 hidden units. To keep training stable, mixture-of-experts normalises the capacity factor before it is combined with the auxiliary loss.

## 6.4 Experiments

On a translation suite, mixture-of-experts improves over switch routing by 6.9 points. Variance across seeds stays below 0.5 points for every configuration we report. Ablating the expert lowers the a reasoning suite score by 2.7 points, the largest drop among all components. With the load balance frozen, mixture-of-experts still recovers most of its improvement on WebMix perplexity.

Variance across seeds stays below 0.6 points for every configuration we report. Throughput is measured with a batch size of 256 on a single accelerator. Mixture-of-experts scores 41.4% on a translation suite, compared with 86.7% for a top-1 router. Ablating the capacity factor lowers the WebMix perplexity score by 1.1 points, the largest drop among all components.

With the shared expert frozen, mixture-of-experts still recovers most of its improvement on a translation suite. Variance across seeds stays below 0.2 points for every configuration we report. Throughput is measured with a batch size of 16 on a single accelerator. The gains of mixture-of-experts grow with the input length, as the expert parallelism becomes the bottleneck.

Ablating the token dropping lowers the WebMix perplexity score by 3.7 points, the largest drop among all components. With the token dropping frozen, mixture-of-experts still recovers most of its improvement on WebMix perplexity. On a translation suite, mixture-of-experts improves over a dense model by 5.1 points. Variance across seeds stays below 0.9 points for every configuration we report.

## 6.5 Summary

A limitation of mixture-of-experts is that the gating network must be tuned separately for each model size. Our analysis covers models up to 1.3B parameters, and larger models may behave differently. The capacity factor also makes mixture-of-experts easier to inspect, since its behaviour can be read off per token. Combining mixture-of-experts with methods that target the token dropping is a promising direction.

Our analysis covers models up to 1.3B parameters, and larger models may behave differently. The auxiliary loss also makes mixture-of-experts easier to inspect, since its behaviour can be read off per token. A limitation of mixture-of-experts is that the expert parallelism must be tuned separately for each model size. The results suggest that the auxiliary loss is the main lever for quality at a fixed budget.

## 7 Conclusion

Across six chapters the same pattern holds: models that choose what to write, through gates, surprise, eviction scores or routing, beat larger memories that store everything. We therefore recommend reporting the write policy of a memory mechanism alongside its size.

## References

[1] C. Castillo and K. Kowalski. Replay buffer for long-term memory models. In Proceedings of the Learning Systems Conference, 2023.

[2] P. Petrova and N. Nakamura. Retrieval key for long-term memory models. In Proceedings of the Learning Systems Conference, 2018.

[3] J. Jansen and V. Varga. Test-time update for long-term memory models. arXiv preprint, 2016.

[4] C. Castillo and J. Jansen. Memory capacity for long-term memory models. arXiv preprint, 2023.

[5] K. Kowalski and V. Varga. Associative loss for long-term memory models. arXiv preprint, 2021.

[6] I. Ivanova and O. Okoro. Associative loss for long-term memory models. In Proceedings of the Sequence Modelling Workshop, 2019.

[7] F. Fujita and U. Ueda. Test-time update for long-term memory models. In Proceedings of the Sequence Modelling Workshop, 2017.

[8] E. Eriksen and Q. Quinn. Segment summary for long-term memory models. In Proceedings of the Sequence Modelling Workshop, 2019.

[9] E. Eriksen and A. Adeyemi. Memory slot for long-term memory models. In Proceedings of the Sequence Modelling Workshop, 2017.

[10] F. Fujita and L. Laurent. Retrieval key for long-term memory models. arXiv preprint, 2024.

[11] Q. Quinn and C. Castillo. Replay buffer for long-term memory models. In Proceedings of the Sequence Modelling Workshop, 2019.

[12] P. Petrova and L. Laurent. Test-time update for long-term memory models. In Proceedings of the Sequence Modelling Workshop, 2021.